

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

FAST-NUCES, Islamabad | Spring 2026

COAL PROJECT BRICK BREAKER

Iteration 3 · Three-Level Edition · x86 Assembly

ALI HAIDER · JEHangIR · TAIMOOR · ABDUL WADOOD

TABLE OF CONTENTS

1. Level Design3

- 1.1 Brick Grid Architecture3
- 1.2 Level 1 - Full Grid3
- 1.3 Level 2 - Corner Gaps4
- 1.4 Level 3 - Sparse Alternating4
- 1.5 Level-Sensitive Paddle Bounce5

2. Bonus System Logic6

- 2.1 Spawn Decision (TrySpawnBonus)6
- 2.2 Bonus Types6
- 2.3 Bonus Movement (UpdateBonus)7
- 2.4 Collection Detection (CheckBonusPaddle)7

3. Game Flow8

- 3.1 Top-Level Navigation8
- 3.2 Game Initialisation8
- 3.3 Per-Level Game Loop (GameLoop)9
- 3.4 Level-Clear and Progression9
- 3.5 End Screens10
- 3.6 Persistent and Reset State10

4. New Features (Iteration 3)11

- 4.1 Pause / Resume11
- 4.2 Mouse Input11
- 4.3 Level Select Screen11
- 4.4 Wide Paddle Bonus12
- 4.5 Ball Trail Effect12
- 4.6 Persistent High Score (score.dat)13
- 4.7 Cycling Bonus Distribution13

1. Level Design

Iteration 3 of Brick Breaker features three separate levels of increasing difficulty. Progress is tracked via the byte variable `currentLevel` (initialised to 1 and incremented by the outer `GameLayoutScreen` loop). Layout and ball velocity per level are applied through `ApplyLevelLayout` and `SetLevelSpeed`, both called from `InitLevel`.

1.1 Brick Grid Architecture

Index calculations: $\text{row} = \text{index} \div 8$, $\text{column} = \text{index} \% 8$. `ApplyLevelLayout` begins by setting all 32 brick states to 1 (active), then zeros selected indices based on the current level and updates the `bricksLeft` counter.

1.2 Level 1 - Full Grid

Active bricks: 32 · `bricksLeft`: 32 · Initial velocity: $\text{DX} = +2$, $\text{DY} = -2$

The opening level presents the maximum number of bricks at minimum ball speed. No gaps in the grid.

Col	0	1	2	3	4	5	6	7
Row 0	#	#	#	#	#	#	#	#
Row 1	#	#	#	#	#	#	#	#
Row 2	#	#	#	#	#	#	#	#
Row 3	#	#	#	#	#	#	#	#

= brick, . = removed

1.3 Level 2 - Corner Gaps

The outermost brick of each end of every row is removed - indices 0, 7, 8, 15, 16, 23, 24, 31. Active bricks: 24. Initial velocity: $\text{DX} = +2$, $\text{DY} = -3$ (greater vertical speed than Level 1).

Col	0	1	2	3	4	5	6	7
Row 0	.	#	#	#	#	#	#	.
Row 1	.	#	#	#	#	#	#	.
Row 2	.	#	#	#	#	#	#	.
Row 3	.	#	#	#	#	#	#	.

1.4 Level 3 - Sparse Alternating

A checkerboard-style sparse matrix. Twelve indices are cleared - 1, 3, 5, 7, 8, 10, 12, 14, 17, 19, 21, 23 - leaving `bricksLeft` = 20. Ball starts at maximum speed: $\text{DX} = +3$, $\text{DY} = -3$. The gap pattern makes bounce trajectories harder to predict.

Col	0	1	2	3	4	5	6	7
Row 0	#	.	#	.	#	.	#	.
Row 1	.	#	.	#	.	#	.	#

Row 2	#	.	#	.	#	.	#	.
Row 3	#	#	#	#	#	#	#	#

1.5 Level-Sensitive Paddle Bounce

Bounce speed off the paddle is not constant; it varies with the current level and the zone of the paddle that was struck. Five zone procedures are used: BounceFarLeft, BounceMidLeft, BounceCenter, BounceMidRight, BounceFarRight.

Paddle Zone	Level 1	Level 2	Level 3	px Range
Far Left	-2, -2	-3, -2	-3, -3	0 – 11
Mid Left	-2, -2	-2, -3	-2, -3	12 – 23
Centre	0, -2	0, -3	0, -3	24 – 35
Mid Right	+2, -2	+2, -3	+2, -3	36 – 47
Far Right	+2, -2	+3, -2	+3, -3	48+

The DX/DY values shown are applied as signed magnitudes - the sign of each component is determined by the zone, and the magnitude is scaled by level.

2. Bonus System Logic

The bonus system randomly spawns a falling power-up whenever a brick is destroyed. Only one bonus may be active at any time. The system comprises four functions: TrySpawnBonus, UpdateBonus, CheckBonusPaddle, and ApplyBonus.

2.1 Spawn Decision (TrySpawnBonus)

TrySpawnBonus is invoked immediately after each brick destruction inside CheckBrickCollision. It uses a lightweight pseudorandom algorithm driven by the byte variable randSeed, which is incremented every frame by UpdateBall.

Spawn gate: if bonusActive = 1 the function returns immediately - no second bonus can spawn while one is already falling.

Spawn probability check:

- › randSeed = randSeed + brickIndex + currentLevel
- › If (randSeed AND 03h) $\neq$ 0 $\rightarrow$ no spawn (75 % rejection rate)
- › If (randSeed AND 03h) = 0 $\rightarrow$ spawn bonus (25 % chance)

On successful spawn the bonus pixel is positioned at the centre of the broken brick (brickHitX + 11, brickHitY + 10); bonusActive is set to 1; bonusType is assigned via a cycling counter described in Section 4.7.

2.2 Bonus Types

Four bonus types are defined. The bonus is rendered as an 8×6 px rectangle whose colour identifies its type.

Type	Colour	Effect	bonusType
1	Green (10)	Slow Ball - sets DX , DY = 2; direction preserved.	1
2	Red (12)	Fast Ball - sets DX , DY = 3; direction preserved.	2
3	Yellow (14)	Extra Life - increments lives (max 9); HUD refreshed immediately.	3
4	Cyan (11)	Wide Paddle - adds 20 px to paddleW (max 100 px).	4

Speed changes preserve the current ball direction by testing the sign of ballDX/ballDY before setting the magnitude (via SetBallDxMagnitude and SetBallDyMagnitude). A ballDX of zero is treated as positive.

The Wide Paddle bonus adds 20 px to paddleW and redraws the playfield via DrawStaticGameFrame, capping expansion at 100 px.

2.3 Bonus Movement (UpdateBonus)

Every frame in which bonusActive = 1, UpdateBonus adds +2 to bonusY. CheckBonusPaddle is then evaluated; if not caught and bonusY exceeds 194, bonusActive is reset to 0 and the bonus is discarded.

No flicker: SavePreviousPositions captures bonusX/Y alongside ball and paddle each frame, enabling DrawGameFrame to erase the previous bonus position with a black fill before redrawing at the new position.

2.4 Collection Detection (CheckBonusPaddle)

CheckBonusPaddle performs a standard AABB test between the 8×6 px bonus rectangle and the paddle. The test is skipped when `bonusActive = 0`. Four axis comparisons are performed:

- › Bonus bottom (`bonusY + 6`) must reach or pass `paddleY`.
- › Bonus top (`bonusY`) must not exceed `paddleY + paddleH`.
- › Bonus right (`bonusX + 8`) must reach or pass `paddleX`.
- › Bonus left (`bonusX`) must not exceed `paddleX + paddleW`.

On overlap, `bonusActive` is set to 0 and `ApplyBonus` is called to deliver the effect.

3. Game Flow

Three nested layers govern game flow: the master loop (menu navigation), GameLayoutScreen (level loop), and GameLoop (per-frame game logic).

3.1 Top-Level Navigation

After the splash screen and name entry, the program loops continuously through the MenuAgain routine. MainMenuScreen passes control to one of four branches based on an index returned in AL:

- › AL = 0 → LevelSelectScreen → GameLayoutScreen
- › AL = 1 → InstructionsScreen
- › AL = 2 → HighScoresScreen
- › AL = 3 → QuitProgram

All screens return to the main menu via MenuAgain. When launching a game, LevelSelectScreen is shown first; the player may choose a starting level or press ESC to return.

3.2 Game Initialisation

Selecting a level triggers GameLayoutScreen, which calls InitGame. InitGame resets score to 0, lives to 3, and currentLevel to the player's choice, then calls InitLevel. InitLevel performs the following for the current level:

- › Resets paddleX to 130, paddleW to 60; ball to X = 158, Y = 150.
- › Resets ball trail positions (trail1–3 X/Y) to ball start.
- › Sets bonusActive = 0, gameDone = 0, endReason = 0.
- › Fills brickStates with 1 for all 32 slots.
- › Calls ApplyLevelLayout to carve level-specific gaps and update bricksLeft.
- › Calls SetLevelSpeed to assign ballDX/ballDY.
- › Calls UpdateLevelText to write the level digit into the HUD string.
- › Calls DrawStaticGameFrame to render the full playfield once.

3.3 Per-Level Game Loop (GameLoop)

GameLoop runs continuously until gameDone becomes non-zero. Each iteration executes the following sequence:

#	Function	Description
1	SavePreviousPositions	Snapshot paddle, ball, bonus X/Y (including 3-frame trail) into prev* variables.
2	ReadGameInput	Non-blocking keyboard and mouse poll. A/D or arrows move paddle; mouse X maps to paddleX; P toggles pause; ESC exits.
3	Guard	If gameDone = 1, exit loop immediately.
4	UpdateBall	Increment randSeed, advance ball by velocity, run wall/paddle/brick/miss collision chain.
5	UpdateBonus	If bonusActive, drop bonus by 2 px, test paddle catch, expire if below playfield.
6	DrawGameFrame	Erase previous positions using trail history, redraw paddle, ball (3-shade trail), and bonus.
7	FrameDelay	~24 ms delay via INT 15h or busy-wait fallback.

8	Repeat	If gameDone = 0, repeat from step 1.
---	--------	--------------------------------------

3.4 Level-Clear and Progression

When bricksLeft reaches 0, CheckBrickCollision sets gameDone = 1 and endReason = 2 (level clear). GameLoopDone evaluates the outcome:

Condition	Behaviour
Level clear (endReason = 2, level < 3)	LevelTransitionScreen is displayed showing the cleared level and current score. Pressing ENTER increments currentLevel and calls InitLevel, which resets level variables while retaining score and lives. GameLoop resumes.
Final win (endReason = 2, level = 3)	endReason is overwritten to 1 and GameOverScreen displays the YOU WIN! screen with the final score.
Game over (endReason = 0)	Lives reached 0 or ESC was pressed. GameOverScreen shows the GAME OVER title with the final score. If the score exceeds highScore, the record is updated and saved to score.dat.

3.5 End Screens

Both terminal conditions (win and game over) share GameOverScreen. The displayed title is chosen by endReason:

endReason	Title Shown	Cause
0	GAME OVER	Lives exhausted or ESC pressed during gameplay.
1	YOU WIN!	All three levels cleared.

After either screen, pressing ENTER returns to the main menu via MenuAgain.

3.6 Persistent and Reset State

Variable(s)	Scope & Reset Behaviour
score	Persists across all levels; never reset between levels.
lives	Persists; decremented on miss, incremented by Extra Life bonus (max 9).
currentLevel	Incremented on level clear; reset to 1 only when a new game begins.
paddleX / ballX / ballY / ballDX / ballDY	Reset per level by InitLevel.
paddleW	Reset to 60 per level; Wide Paddle bonus can expand up to 100 px within a level.
trail1–3 X/Y	Reset to ball start position (158, 150) per level by InitLevel.
brickStates / bricksLeft	Rebuilt per level by ApplyLevelLayout.
bonusActive / bonusType / bonusX / bonusY	Cleared per level (bonusActive = 0).

highScore / highScoreName	Persistent across sessions; saved to score.dat and loaded at startup.
playerName	Set once at startup; never reset.

4. New Features - Iteration 3

The following features were added in Iteration 3 beyond the base game documented in previous iterations.

4.1 Pause / Resume

Pressing P or p at any point during gameplay calls `TogglePause`. Execution enters `PauseLoop`, a tight loop that consumes keyboard input and blocks all game updates and rendering until P or p is pressed again. The game resumes from exactly the same state; no variables are modified during the pause.

The pause key is detected inside `ReadGameInput` after the mouse poll and before the directional key checks, so it responds on the same input cycle as other keyboard commands.

4.2 Mouse Input

`ReadGameInput` polls the mouse driver (INT 33h, function 3) on every frame before checking the keyboard. The returned horizontal cursor position (CX, halved from hardware coordinates) is mapped directly to `paddleX` after clamping:

- › Left clamp: $\text{paddleX} \geq 14$
- › Right clamp: $\text{paddleX} \leq 306 - \text{paddleW}$ (accounts for variable paddle width)

To avoid redundant redraws, the previous mouse X position is stored in `prevMouseX`; paddle movement only occurs when the cursor actually moves. Keyboard input is processed in the same frame regardless.

4.3 Level Select Screen

A new `LevelSelectScreen` is inserted between the main menu and the game. It presents three selectable options:

- › **LEVEL 1 (SLOW)** - DX = +2, DY = -2. Full 32-brick grid.
- › **LEVEL 2 (MED)** - DX = +2, DY = -3. Corner-gap layout.
- › **LEVEL 3 (FAST)** - DX = +3, DY = -3. Sparse checkerboard layout.

Navigation supports keyboard (arrow keys + ENTER), mouse hover (via `prevMouseY` delta detection), and mouse click. ESC from this screen returns to the main menu (AL = 0). The selected index is returned in AL (1–3) and stored in `currentLevel` before `InitGame` is called, letting players begin at any level rather than being forced to start at Level 1.

4.4 Wide Paddle Bonus

A fourth bonus type (`bonusType = 4`, rendered in cyan, colour 11) has been added to the bonus system. When collected, `ApplyBonus` calls `BonusWidePaddle`, which adds 20 px to `paddleW` and clamps the result to a maximum of 100 px. `DrawStaticGameFrame` is then called to immediately re-render the playfield with the wider paddle.

`paddleW` is reset to 60 px at the start of each level by `InitLevel`, so the effect does not carry over between levels. The right-clamp calculation in both `ReadGameInput` and the keyboard movement routines references `paddleW` directly, ensuring the paddle never goes out of bounds regardless of width.

4.5 Ball Trail Effect

A three-frame motion trail is rendered behind the ball to convey speed and direction. Three pairs of variables (`trail1X/Y`, `trail2X/Y`, `trail3X/Y`) store the ball's position for the previous three frames. These are updated in `SavePreviousPositions` using a shift-register pattern:

- › `trail3` ← `trail2`
- › `trail2` ← `trail1`

- › trail1 ← current ballX/Y

DrawBall renders three filled rectangles of the same dimensions as the ball ($\text{ballSize} \times \text{ballSize}$):

- › trail2 position - colour 8 (dark grey, oldest visible trail)
- › trail1 position - colour 7 (light grey, mid trail)
- › current position - colour 15 (white, the ball itself)

ErasePreviousBall erases from the trail3 position (the oldest) so the trail disappears naturally without artefacts. All trail positions are reset to (158, 150) by InitLevel at the start of each level.

4.6 Persistent High Score (score.dat)

A file-based high-score system persists the top score across sessions using DOS interrupt 21h file I/O.

- › **LoadScore** - called at startup. Opens score.dat for reading; if found, reads the stored score word and name string into highScore and highScoreName.
- › **SaveScore** - called from GameOverScreen when the current score exceeds highScore. Creates/truncates score.dat and writes the updated score and player name.

The high-score check in GameOverScreen copies playerName into highScoreName and calls SaveScore before rendering the end screen. HighScoresScreen displays highScoreName and highScoreValue (the formatted five-digit score string) on a dedicated screen accessible from the main menu.

4.7 Cycling Bonus Distribution

In earlier iterations bonusType was assigned via a modulo-3 operation on randSeed, which could in practice never produce bonusType 4 (Wide Paddle). Iteration 3 replaces this with a cycling counter (bonusTypeCounter) that increments on every successful spawn and wraps at 4, guaranteeing round-robin delivery of all four bonus types:

```
bonusTypeCounter = (bonusTypeCounter + 1) mod 4 → bonusType = result + 1 → values cycle 1, 2, 3, 4, 1, 2, 3, 4 ...
```

The 25 % random spawn gate (Section 2.1) is retained so the overall spawn probability is unchanged; only the type assignment is made deterministic. This ensures players encounter all bonus types roughly equally across a session.